Marleena Rose Limbrick
Prof. BJohnson
CMSI 4071
4 November 2026

Assignment 2: Piloni & Miles Reading

Problem 1:

The two major concerns of any software project are functionality and schedule. We think functionality is more important because delivering software on time doesn't mean much if it doesn't actually work or meet the customer's needs. However, these concerns can sometimes conflict because adding more features or improving existing ones can take additional time, potentially delaying the project.

Problem 2:

The five main phases of Agile development are requirements, design, development, testing, and deployment. We don't think all five phases need to be completed at the beginning of every iteration, since Agile is meant to be flexible and allows teams to make changes throughout development. We also don't think we should save all of our work for the end of the project because delivering working software throughout the process allows customers to provide feedback and helps developers catch problems earlier (duh!).

Problem 3:

The main phases of Waterfall development are requirements, design, implementation, testing, deployment, and maintenance. Unlike Agile, Waterfall follows a more sequential process where each phase is generally completed before moving to the next. Waterfall also includes more distinct stages for planning and maintaining the software. We think these additional phases can be useful because they encourage developers to plan ahead and consider what happens after the software is released. For example, a maintenance phase would be necessary if a company needed to fix bugs or update software after customers had already started using it.

Problem 4:

- ***User story:*** A short description of a feature from the perspective of the person who will use it.
- ***Blueskying:*** Brainstorming ideas and possibilities without immediately worrying about limitations.
- ***Four things user stories SHOULD do:*** Be understandable, provide value to the customer, be small enough to estimate, and be testable.
- ***Three things user stories SHOULD NOT do:*** Be overly complicated, contain unnecessary technical details, or be too large and vague.
- ***Does Waterfall have user stories?*** Waterfall typically relies on detailed requirements documents rather than Agile-style user stories, although user stories can still be used.

Problem 5:

Statement 1: "All assumptions are bad, and no assumption is a 'good' assumption."

- We disagreed because assumptions aren't always bad, especially when they're based on previous experience or reasonable expectations. However, assumptions can become a problem when developers rely on them without confirming what the customer actually wants. We think it's important to verify assumptions whenever possible to avoid misunderstandings later.

Statement 2: "A 'big' user story estimate is a 'bad' user story estimate."

- We somewhat agree because a large estimate can mean that a user story is too complicated or contains too many different tasks. This can make it harder to accurately estimate how much time the work will take. We think breaking larger user stories into smaller ones makes the work easier to manage and helps the team track its progress.

Problem 6:

You can dress me up as a use case for a formal occasion: User Story
The more of me there are, the clearer things become: User Story
I help you capture EVERYTHING: Blueskying, Observation
I help you get more from the customer: Role Playing, Observation
In court, I'd be admissible as firsthand evidence: Observation
Some people say I'm arrogant, but really I'm just about confidence: Estimate
Everyone's involved when it comes to me: Blueskying

We agree with the textbook's answers because each technique helps developers better understand the customer's requirements. For example, blueskying allows everyone to contribute ideas, while observation helps developers understand how customers actually use a system. We also agree that planning poker doesn't involve everyone because customers typically aren't part of the estimation process.

Problem 7:

A "better than best-case" estimate means that the work is expected to be completed even faster than the most optimistic realistic estimate. This can be a problem because it creates unrealistic expectations and doesn't account for unexpected issues that might come up during development.

Problem 8:

We think the best time to tell a customer that we won't meet the delivery schedule is as soon as we realize there's a problem. This gives the customer time to adjust their expectations and allows us to discuss possible solutions. We think it would be a difficult conversation because the customer might be frustrated or disappointed. It would be even more difficult if we waited until

the deadline was approaching or had already passed, especially if we knew about the delay beforehand.

Problem 9:
We think branching in software configuration management is generally a good thing because it allows developers to work on different features or fixes without interfering with the main version of the project. However, having too many branches can make it difficult to keep track of changes and merge everyone's work together. For example, if two developers are working on separate branches and both modify the same file, they might run into merge conflicts when combining their changes.

Problem 10:
We've used Git and GitHub for version control in my development projects. One thing we like about Git is that it allows me to track changes, work on different branches, and return to previous versions of my code if something goes wrong. GitHub also makes it easier to collaborate with other developers on the same project. However, one thing we dislike is that resolving merge conflicts can sometimes be confusing, especially when multiple people have modified the same files.